

CS5344 PROJECT PROPOSAL — COVERAGE-AWARE NORMAL AUGMENTATION

CHEN YICHEN
A0328816W

KONG JIE
A0319067Y

WU YIJUN
A0318466W

chenyichen@u.nus.edu

e1520927@u.nus.edu

e1520326@u.nus.edu

Team 27: Offer-Squad
27 September 2026

ABSTRACT

We study normal-data augmentation for network anomaly detection under a fixed-size submission protocol. The task uses NSL-KDD and UNSW-NB15 and evaluates generated normal data with fixed ECOD and Isolation Forest detectors. For development and ablation, we use $N_{\text{dev}} = 2,000$ intended-normal rows as a matched pilot budget. The final Kaggle submission must contain exactly $N_{\text{sub}} = 70,000$ rows. Our core generator combines normal-mode quota allocation, compatible local interpolation, and bootstrap support retention. We also test a known-anomaly proximity screen as an optional regularizer. Its effect is evaluated through matched detector-level ablations and a proxy test on withheld anomaly clusters.

Index Terms— network anomaly detection, data augmentation, tabular synthesis, ECOD, Isolation Forest

1. TASK, MOTIVATION, AND RESEARCH QUESTION

The project asks us to generate a normal-only training set for two network intrusion benchmarks, NSL-KDD and UNSW-NB15. NSL-KDD was introduced to reduce important redundancy problems in KDD CUP 99 [1]. UNSW-NB15 was built from modern normal activities and attack behaviours and provides a more recent traffic setting [2]. For each dataset, let $D_0 = \{x_i : y_i = 0\}$ denote released normal records and $D_1 = \{x_i : y_i = 1\}$ denote released anomalies. We use $N_{\text{dev}} = 2,000$ normal rows as a development budget for matched local comparisons. The final Kaggle file must contain exactly $N_{\text{sub}} = 70,000$ rows. The generator is parameterized by output size N , so the same learned proportions and constraints can be used at either scale.

The course score averages the AUPRC of two fixed anomaly detectors,

$$S(\tilde{D}) = \frac{1}{2} \left[\text{AUPRC}_{\text{ECOD}}(\tilde{D}) + \text{AUPRC}_{\text{IF}}(\tilde{D}) \right]. \quad (1)$$

ECOD estimates empirical tail probabilities feature by feature and aggregates them into an outlier score [10]. Isolation Forest separates unusual points through short random partition paths [11]. Since their inductive biases differ, an augmentation strategy that helps one detector may not help the other. The submitted distribution therefore needs to preserve broad normal support while avoiding synthetic artifacts that alter tails, density, or isolation structure.

The supplied baseline samples a small set of normal prototypes, expands them by bootstrap, and adds independent numeric noise. This is simple and reproducible, but a small prototype pool can omit low-frequency normal modes. If a mode has population fraction p , the probability that none of m prototype draws comes from that mode is

approximately $(1 - p)^m$. With $m = 20$ and $p = 0.05$, this omission probability is $0.95^{20} = 35.8\%$. This is an illustration rather than a measured property of either dataset.

Our primary research question is whether controlled coverage of normal modes and locally constrained synthesis can improve both fixed anomaly detectors while scaling from a matched 2,000-row development budget to the required 70,000-row submission. We also examine whether screening against observed anomalies provides transferable benefit without degrading performance on withheld anomaly clusters. Each generated row has an auditable origin, and optional components are assessed with matched ablations.

2. PRELIMINARY DATA AUDIT AND EXPLORATORY ANALYSIS

We performed an initial audit on the released training files before designing the generator. The NSL-KDD file contains 30,000 rows, including 20,000 normal and 10,000 anomalous records. The UNSW-NB15 file contains 50,000 rows, including 35,000 normal and 15,000 anomalous records. Neither dataset contains missing or infinite values, and no exact duplicate occurs among the normal records. These checks show that imputation and duplicate removal are not central preprocessing steps, so the augmentation design can focus on support coverage and feature dependence.

The normal data show substantial discrete heterogeneity. NSL-KDD contains 69 normal categorical signatures, while UNSW-NB15 contains 29. These signatures are defined by the three categorical fields in each dataset. To test whether a 20-prototype pool represents this diversity, we sampled 20 normal records without replacement for 10,000 seed-42 trials. The prototypes covered only $9.5\% \pm 2.0\%$ of the NSL-KDD signatures and $23.7\% \pm 4.5\%$ of the UNSW-NB15 signatures on average. This result supports the coverage motivation for our method.

The two-dimensional visualizations show the same pattern from another perspective. In Fig. 2, baseline-generated NSL-KDD samples are concentrated around a limited subset of the full normal support. Fig. 3 shows a similar concentration pattern on UNSW-NB15. Because t-SNE does not preserve all global distances, we use these plots as qualitative evidence and rely on the measured signature coverage for the quantitative comparison. Together, the two analyses motivate explicit normal-mode allocation instead of a small prototype pool.

The normal feature space also contains strong dependence and scale imbalance. In NSL-KDD, Spearman correlation is 0.941 for `error_rate` versus `srv_error_rate` and -0.927 for `same_srv_rate` versus `diff_srv_rate`. In UNSW-NB15, it is 0.988 for `tcp_rtt` versus `synack` and 0.957 for `spkts` versus `dpkts`. The right panels of Figs. 2 and 3 further show representa-

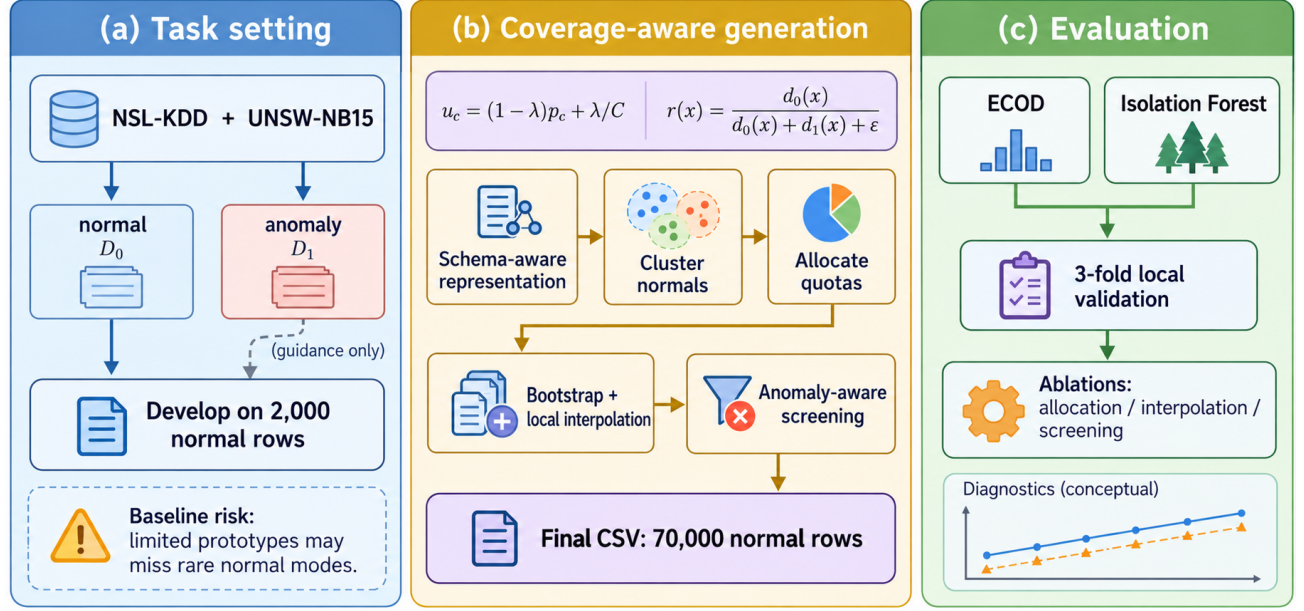


Fig. 1. Overview of the proposed study. Panel (a) distinguishes the matched 2,000-row development budget from the final 70,000-row Kaggle export shown in panel (b). The anomaly-aware screen is an optional branch evaluated against a gate-off core. Panel (c) presents fixed-detector evaluation and ablations.

Table 1. Measured statistics from the released training data. Prototype coverage is the mean $\pm$ standard deviation over 10,000 seed-42 random 20-prototype samples without replacement.

Dataset	Rows	Normal	Anom.	Signatures	20-proto. cov.
NSL-KDD	30,000	20,000	10,000	69	9.5% $\pm$ 2.0%
UNSW-NB15	50,000	35,000	15,000	29	23.7% $\pm$ 4.5%

tive heavy-tailed numerical features. These observations motivate shared-coefficient local interpolation together with the $\log(1+x)$ preprocessing used for nonnegative count and volume features.

The audit provides empirical input for the generator design. The released data already show that a 20-record prototype set represents only a small fraction of observed normal categorical states, while several numerical features are strongly dependent. During model selection, we will continue to monitor normal-only quantiles, zero frequencies, category frequencies, duplicate rates, tail mass, and correlation error as diagnostics of the generated data.

The two datasets will continue to use separately fitted statistics. They share only the algorithmic recipe and validation logic. All pre-processing parameters used during model selection will be estimated within the current development-training partition, and private validation or test rows will never be used to fit a transformation, cluster, neighbor index, or screening threshold.

3. RELATED WORK AND POSITIONING

Local resampling. SMOTE generates synthetic observations by interpolating between nearby samples [3]. ADASYN changes the number of generated points according to local learning difficulty [4]. These methods motivate two decisions that are useful here: where to spend the fixed sample budget and how to create a new point once a region is selected. Their original goal is supervised class

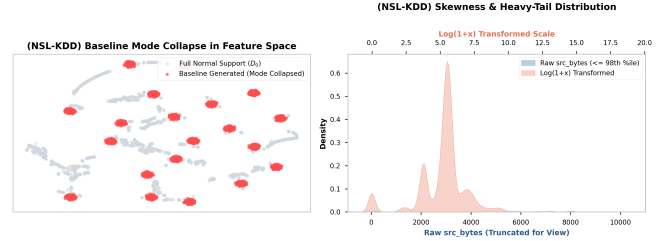


Fig. 2. Exploratory analysis of NSL-KDD. Left: a t-SNE projection shows that samples produced by the supplied prototype-based baseline are concentrated in a limited subset of the full normal support D_0 . Right: the raw `src_bytes` distribution is strongly skewed and heavy-tailed, while the $\log(1+x)$ transform compresses its dynamic range.

imbalance. Our output constraint is different because every submitted row must represent normality. We therefore apply allocation only within D_0 and use anomalies as a rejection signal rather than as a class to oversample.

Learned tabular generation. CTGAN uses conditional generation and specialized representations for mixed-type tabular data [5]. More recent diffusion methods show that mixed numerical and categorical data can be modeled with greater flexibility. TabDDPM handles heterogeneous tabular features in a diffusion framework [6]. CoDi couples continuous and discrete diffusion processes [7]. TabSyn performs diffusion in a learned latent space [8], while TabDiff uses a joint mixed-type diffusion process with feature-wise schedules [9]. These models provide useful comparators under the fixed detectors, matched development budget, and 70,000-row submission scale.

Positioning. Our contribution is an interpretable combination of

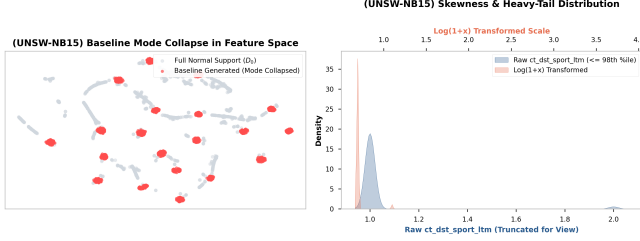


Fig. 3. Exploratory analysis of UNSW-NB15. Left: baseline-generated samples occupy only part of the observed normal support in the t-SNE projection. Right: the raw `ct_dst_sport_ltm` distribution shows strong skew and scale concentration, which are reduced after the $\log(1+x)$ transform.

normal-mode allocation, compatible local interpolation, and bootstrap support. A known-anomaly proximity screen is tested separately. We compare each component with all-normal bootstrap and the supplied jitter baseline. Precision-recall evaluation is appropriate for imbalanced anomaly detection because it keeps positive-class prevalence visible in the metric [12].

A second distinction is utility versus fidelity. Generative-model papers commonly evaluate statistical similarity, predictive utility, or both. Our primary objective is narrower. The generated table is an input to two fixed unsupervised detectors, so downstream detector performance is the deciding criterion. Distributional diagnostics are still useful because they explain why a method behaves differently, but they do not replace AUPRC. We will therefore use CTGAN and TabDDPM as matched-budget reference points for comparison.

The proposal also differs from standard class-imbalance augmentation. We do not seek a balanced training set and we do not synthesize anomalies. The task instead resembles support design for one-class or unsupervised detection. This makes over-expansion of normal support a serious risk. If synthetic rows fill regions that should remain sparse, both ECOD and Isolation Forest may learn an overly broad notion of normality. The allocation cap, compatible-neighbor rule, interpolation bound, and bootstrap mixture constrain this risk without requiring anomaly labels.

4. INTENDED APPROACH AND JUSTIFICATION

4.1. Overview and shared representation

Figure 1 summarizes the workflow. Normal records determine the generation support. Observed anomalies do not contribute output rows. They appear only in an optional proximity screen for newly interpolated candidates. Local validation selects the configuration before the final 70,000-row generator is refitted on all released training data.

For distance calculations we will use a mixed representation. Nonnegative count and volume features are transformed with $\log(1+x)$. Numerical columns are centered by the normal median and scaled by the interquartile range. A nonzero standard deviation is used when the IQR is zero. Constant columns are excluded from distances and restored during generation. Nominal variables are one-hot encoded. Numerical and discrete blocks are normalized by the square root of their dimensions so that one large block does not dominate by size alone. Submitted rows remain in original units.

4.2. Allocate the fixed budget across normal modes

We partition normal development records into C groups using seeded mini-batch k -means in the mixed representation. These groups approximate normal modes. They are not attack categories. Let $p_c = |D_0^{(c)}|/|D_0|$ be the empirical mass of group c . We blend the empirical distribution with a uniform coverage target,

$$u_c = (1 - \lambda)p_c + \lambda/C. \quad (2)$$

To avoid strongly inflating a tiny group, the final weight is capped relative to its empirical mass. We use

$$q_c = \min\{\kappa p_c, t u_c\}, \quad \sum_{c=1}^C q_c = 1, \quad (3)$$

with $\kappa = 4$ and scalar t found by bisection. Largest-remainder rounding turns Nq_c into integer quotas that sum exactly to the chosen output size N . We use $N = N_{\text{dev}} = 2,000$ for matched development experiments and then set $N = N_{\text{sub}} = 70,000$ for the final Kaggle submission. Because the allocator is defined through proportions q_c , this scale change affects only the realized integer quotas, not the allocation rule itself. The case $\lambda = 0$ is a required control because it recovers empirical allocation up to rounding.

4.3. Generate within compatible local neighborhoods

For an anchor x_i in group c , eligible donors are chosen within the same cluster to preserve discrete operational states while mitigating severe fallback caused by rare category combinations. Specifically, we employ a **two-stage donor selection mechanism**:

1. **Strict Match:** Donors must share the exact categorical and binary signature with anchor x_i .
2. **Relaxed Match:** If strict donors within group c are fewer than k , we relax only secondary binary indicators. All three categorical fields remain identical: `protocol.type`, `service`, and `flag` for NSL-KDD; `proto`, `service`, and `state` for UNSW-NB15.

When relaxed donors remain insufficient, we fall back to bootstrap. Configurations with fallback rates above 15% are flagged as unreliable. We select donor x_j uniformly from the k nearest eligible neighbors in the mixed representation space (excluding x_i). For the non-binary numeric block R and discrete block C , generation uses

$$\tilde{x}_R = (1 - \alpha)x_{i,R} + \alpha x_{j,R}, \quad \tilde{x}_C = x_{i,C}, \quad (4)$$

where $\alpha \sim U(0, a)$ with $a \leq 0.5$. Restricting a limits departure from the anchor. Within quota N_c , we generate $N_c^{\text{boot}} = \lfloor \rho N_c \rfloor$ rows by bootstrap and $N_c - N_c^{\text{boot}}$ by interpolation, where ρ is the bootstrap share. Integer counts are rounded and legal domain bounds are enforced.

4.4. Optional known-anomaly proximity screening

The coverage allocator, compatible interpolation, and bootstrap mixture form the core generator. We additionally test a screen based on observed anomalies. This screen is treated as an empirical regularizer because private anomalies may follow unseen attack mechanisms. Let $T(x)$ be the fitted representation and let $d_b(x)$ denote nearest-neighbor distance from candidate x to class $b \in \{0, 1\}$. We define

$$r(x) = \frac{d_0(x)}{d_0(x) + d_1(x) + \varepsilon}, \quad \varepsilon = 10^{-8}. \quad (5)$$

A high $r(x)$ means the candidate is relatively closer to observed anomalies than to normal support. To avoid self-distance leakage, each normal fitting record is calibrated against its nearest *other* normal record. We set τ to the 95th percentile of these leave-one-out scores. Candidates with $r(x) > \tau$ or unresolved schema violations are rejected. The gate applies only to interpolated candidates and leaves the bootstrap component unchanged.

The distance ratio is evaluated through matched gate-off and gate-on experiments with detector scores reported separately. We keep the screen when it improves or matches mean local AUPRC without materially degrading the withheld-anomaly-cluster stress test or either detector. Otherwise, the final configuration uses the gate-off variant.

This design keeps each output row auditable. Every accepted row can be traced to a group, anchor, donor, mixing coefficient, and optional screening decision. The final CSV contains newly assigned row IDs from 0 to 69,999 followed by features in the released order; the development code retains a separate audit log.

5. EXPERIMENTAL DESIGN AND EXPECTED OUTCOMES

Leakage-controlled development. Each released training set will first be split into an 80% development portion and a 20% untouched local audit portion. Splits will be stratified by the binary label when feasible. Exact duplicate feature rows will be grouped so that identical rows do not cross the split. Three folds inside the development portion will be used for parameter selection. Every transformation, cluster, neighbor index, and screening threshold will be fitted on the training fold only. After selecting a configuration, we will refit on the development portion and evaluate the local audit set once. The final Kaggle generator will then be refitted on all released training data.

Matched baselines. Local model selection will use the same $N_{\text{dev}} = 2,000$ normal-row budget for every method so that comparisons remain controlled and computationally manageable. The mandatory baselines are the supplied 20-prototype jitter method, bootstrap from all normal rows, all-normal bootstrap with the same global jitter, and compatible local interpolation without coverage reweighting or anomaly guidance. We will also run normal-only CTGAN and TabDDPM when the compute budget permits. After the configuration is frozen, the selected generator will be rerun at the Kaggle submission scale of 70,000 rows. Learned baselines will use declared versions, training budgets, and random seeds. The simple controls provide the reference point for evaluating the added method components.

Unknown-anomaly stress test. The released labels do not identify semantic attack types, so a true leave-one-attack-type-out study is impossible from binary labels alone. As a limited proxy, we will cluster development anomalies into up to five groups. For each group in turn, the optional screen is fitted without that group; detectors trained on the resulting synthetic normals are then scored on the withheld group together with the same held-out normal rows. We will report mean and worst-group AUPRC for this proxy test.

Search space and reproducibility. The initial search uses $C \in \{8, 16, 32\}$, $k \in \{5, 10\}$, $\lambda \in \{0, 0.15, 0.30\}$, $a \in \{0.1, 0.3, 0.5\}$, and $\rho \in \{0.5, 0.8\}$. Before inspecting detector scores, we will freeze in a configuration file 12 tuples spanning every level of each parameter. Generation will be repeated with seeds 17, 42, and 83. Detector scores will be reported separately before their mean. We will also report schema violations, exact duplicates, mode coverage, numerical correlation error, runtime, and peak memory.

Expected outcomes and decision rule. We expect the strongest gains, if any, from using the full normal pool and reducing omission

Table 2. Planned ablations and the question each one answers.

Comparison	Question and diagnostics
Empirical vs. coverage allocation	Does broader mode coverage help? Report quotas, category coverage, and q_c/p_c .
Jitter vs. local interpolation	Does local structure matter? Report correlation error and tail quantiles.
Gate off vs. gate on	Does anomaly guidance transfer? Report rejection rate and held-out anomaly-cluster scores.
Pure interpolation vs. mixture	Is observed support retained? Report duplicates and signature coverage.
20 prototypes vs. full normal pool	Is the small pool a bottleneck? Report mode omission and score variability.

of legitimate low-frequency modes. Local interpolation may preserve dependence better than independent jitter, while bootstrap protects observed tails and discrete patterns. The optional screen may reject bridges near known anomalies, but it may also overfit their geometry. We retain it when gate-on improves or matches mean local AUPRC, causes no material reduction in worst-group proxy-unseen AUPRC, and does not substantially degrade either detector. Otherwise, the gate-off core remains the final method. Success for the primary hypothesis means reproducible improvement over the supplied baseline and all-normal bootstrap on both datasets.

Metric handling. We will follow the official preprocessing and score aggregation once the course evaluator is released. Before that point, average precision will be used only as a local proxy for area under the precision-recall curve. We will keep the metric implementation consistent across experiments. For every experiment we will keep ECOD and Isolation Forest scores separate and report their mean only after the individual values are shown. This matters because a change that improves empirical tail coverage can affect ECOD differently from a change that alters random partition geometry in Isolation Forest.

For generator variability, each selected method will be run with seeds 17, 42, and 83. If the official Isolation Forest evaluator uses multiple fixed seeds, that detector randomness will be kept separate from generator randomness. On the frozen local audit set we will compute paired bootstrap intervals for score differences using groups of identical feature rows as the resampling unit. These intervals describe local sampling uncertainty on the frozen audit set.

Failure analysis. A higher local AUPRC is not sufficient if it comes with obvious data defects. We will therefore inspect every competitive configuration for invalid categorical values, illegal numeric ranges, unresolved identities, exact duplicate concentration, rare-signature collapse, and large correlation error. We will also compare the fraction of generated rows that came from fallback bootstrap. A high fallback rate would indicate that the compatible-neighbor rule is too restrictive or that clustering fragmented the data. A high rejection rate concentrated in one group would indicate that the anomaly screen is interacting strongly with the coverage allocator.

Planned visual diagnostics. The final report will pair detector scores with signature coverage, tail quantiles, dependence error, mixture composition, and seed variability. These plots will help distinguish gains from broader coverage from detector-specific effects,

while AUPRC remains the main utility metric.

Milestones. The first implementation stage completes the data audit and reproduces the supplied baseline together with all-normal bootstrap. The second stage implements the shared representation, clustering, quota allocation, and compatible local synthesis. The third stage runs core ablations and tests the optional anomaly screen. CTGAN and TabDDPM are run after the lightweight pipeline is stable. The final stage freezes the selected configuration, reruns all seeds, produces the two 70,000-row submission CSV files, and packages code plus audit logs.

Expected deliverable. The project will produce more than a leaderboard entry. The final report will explain whether normal-mode coverage and local dependence preservation help the two fixed detectors under a matched 2,000-row development budget, whether optional anomaly guidance transfers beyond the groups used to fit it, and whether the selected configuration remains well behaved at the required 70,000-row Kaggle scale. The final method will use the simplest reproducible variant supported by the experiments.

Threats to validity and mitigation. Parameter selection uses the released data, while the final score is computed on a hidden evaluation split. We therefore keep the search space small and apply the same repeated validation protocol to all configurations. Binary labels do not reveal fine-grained attack families, so the anomaly-cluster stress test serves as a proxy analysis for transfer beyond the observed groups. We also report per-detector scores, duplicate rates, and support coverage to distinguish distributional changes from detector-specific effects.

Practical feasibility. The core pipeline is deliberately lightweight. Clustering, neighbor search, quota calculation, and interpolation use standard Python libraries and should run on a laptop for the released datasets. This keeps most of the effort on the research question rather than on engineering and tuning. The learned baselines are therefore optional reference points rather than the centre of the project. The lightweight controls remain the main basis for testing whether broader normal coverage helps under the course metric, while CTGAN and TabDDPM serve as additional reference points when the compute budget permits.

Submission and reporting criteria. We will freeze the final generator after three checks. The chosen variant should outperform or match the supplied baseline on both datasets under the local protocol, satisfy the data-quality checks, and show stable performance across seeds. The final report will include the fixed configuration, all ablation results, the two generated CSV files, and enough implementation detail for another student to reproduce the pipeline from the released training data.

Team workflow. To keep the implementation organized, we will separate the code base into four parts: data audit, generator, evaluation, and reporting. The audit module will produce feature summaries and schema checks. The generator module will implement the quota allocator, local interpolation, bootstrap fallback, and anomaly-aware screen. The evaluation module will handle local splits, detector training, and ablations. The reporting module will create the summary tables and plots used in the final write-up. This structure reduces debugging overhead and makes it easier to trace score changes to individual components.

6. REFERENCES

- [1] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” in *Proc. IEEE CISDA*, 2009, pp. 1–6, doi: 10.1109/CISDA.2009.5356528.
- [2] N. Moustafa and J. Slay, “UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set),” in *Proc. MilCIS*, 2015, pp. 1–6, doi: 10.1109/MilCIS.2015.7348942.
- [3] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002, doi: 10.1613/jair.953.
- [4] H. He, Y. Bai, E. A. Garcia, and S. Li, “ADASYN: Adaptive synthetic sampling approach for imbalanced learning,” in *Proc. IEEE IJCNN*, 2008, pp. 1322–1328, doi: 10.1109/IJCNN.2008.4633969.
- [5] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, “Modeling tabular data using conditional GAN,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [6] A. Kotelnikov, D. Baranchuk, I. Rubachev, and A. Babenko, “TabDDPM: Modelling tabular data with diffusion models,” in *Proc. ICML*, PMLR 202, 2023, pp. 17564–17579.
- [7] C. Lee, J. Kim, and N. Park, “CoDi: Co-evolving contrastive diffusion models for mixed-type tabular synthesis,” in *Proc. ICML*, PMLR 202, 2023, pp. 18940–18956.
- [8] H. Zhang *et al.*, “Mixed-type tabular data synthesis with score-based diffusion in latent space,” in *International Conference on Learning Representations*, 2024.
- [9] J. Shi, M. Xu, H. Hua, H. Zhang, S. Ermon, and J. Leskovec, “TabDiff: A mixed-type diffusion model for tabular data generation,” in *International Conference on Learning Representations*, 2025.
- [10] Z. Li, Y. Zhao, X. Hu, N. Botta, C. Ionescu, and G. H. Chen, “ECOD: Unsupervised outlier detection using empirical cumulative distribution functions,” *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12181–12193, 2023, doi: 10.1109/TKDE.2022.3159580.
- [11] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proc. IEEE ICDM*, 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17.
- [12] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets,” *PLOS ONE*, vol. 10, no. 3, e0118432, 2015, doi: 10.1371/journal.pone.0118432.